SVM Dual Formulation and the Kernel Trick

Machine Learning — Lecture 7

Index

- 1. Recap of the SVM Setup — *Where we left our margin-maximizing hero.*
- 2. The Lagrangian of the Separable SVM — *Turning handcuffs (constraints) into price tags (penalties).*
- 3. Stationarity Conditions — *Differentiate, set to zero, and let the formulas confess.*
- 4. From Primal to Dual — *The great substitution: θ and b vanish, only α survives.*
- 5. The Dual Problem in Matrix Form — *Hand it to any quadratic-program solver and go for tea.*
- 6. Non-Separable Classes with Slack — *Forgiving a few rebels without losing the plot.*
- 7. Recovering Parameters and Predicting — *Getting θ, b back from α and classifying new points.*
- 8. When a Line Is Not Enough — *Rings, XOR, and other things a ruler cannot split.*
- 9. Lifting Data to Higher Dimensions — *If 2-D won't cooperate, add a dimension.*
- 10. Kernel Functions — *Similarity scores with a fancy name.*
- 11. The Kernel Trick — *Getting the 6-D answer while never leaving 2-D.*
- 12. Kernels Inside the SVM Dual — *One tiny swap, infinite power.*
- 13. The Gaussian Kernel — *Similarity that fades with distance.*
- 14. Practice Questions — *Five puzzles to keep your brain honest.*

1. Recap of the SVM Setup

We are given a training set of n labelled points:

$$S = \{(x_1, y_1), \dots, (x_n, y_n)\}, \qquad y_i \in \{+1, -1\}$$

The classifier is

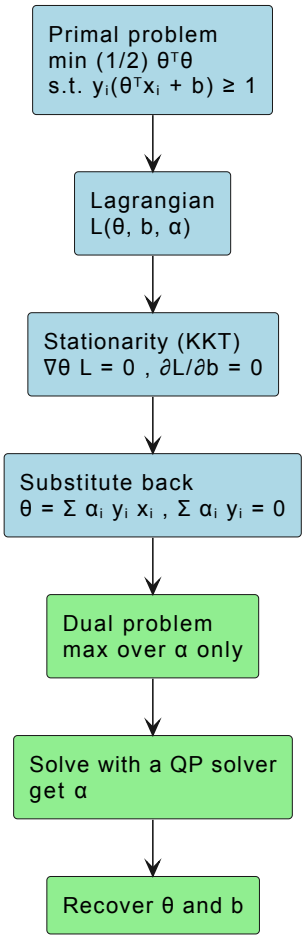
$$h(x) = \text{sgn}(\theta^T x + b)$$

If $h(x) = +1$ the point is assigned to the positive class, and if $h(x) = -1$ it is assigned to the negative class.

To find θ and b we **maximize the margin** while making sure every training point is classified correctly. For **linearly separable classes** this is the optimization problem (the **primal problem**):

$$\min_{\theta, b} \ \frac{1}{2}\theta^T\theta \quad \text{subject to} \quad y_i(\theta^T x_i + b) \geq 1, \quad \forall\, i \in \{1, \dots, n\}$$

It is a quadratic objective with linear inequality constraints. In the previous lecture we saw that such problems can be handled with the **Lagrange multiplier technique** and the **KKT conditions**. Today we apply that machinery to the SVM.



2. The Lagrangian of the Separable SVM

In the Lagrange multiplier technique we keep the original cost function and add a **penalty** for each constraint. There are n constraints, so there are n multipliers α_i:

$$L(\theta, b, \alpha) = \frac{1}{2}\theta^T\theta - \sum_{i=1}^{n} \alpha_i(y_i(\theta^T x_i + b) - 1)$$

Here α denotes the vector that contains all the α_i. The KKT conditions require

$$\alpha_i \geq 0, \qquad i = 1, \dots, n$$

Recall what the multipliers mean:

- $\alpha_i = 0$ for every **inactive** constraint (the point lies strictly outside the margin).
- $\alpha_i > 0$ only for **active** constraints (the point sits exactly on the margin, i.e. it is a **support vector**).

The modified cost function L is now a function of θ, b and α.

3. Stationarity Conditions

To find the optimum we differentiate L with respect to each of the parameters and set the result to zero. Before differentiating, expand the penalty term:

$$L(\theta, b, \alpha) = \frac{1}{2}\theta^T\theta - \sum_{i=1}^{n} \alpha_i y_i \theta^T x_i - b\sum_{i=1}^{n} \alpha_i y_i + \sum_{i=1}^{n} \alpha_i$$

3.1 Gradient with respect to θ

The first term gives θ. In the second term, $\alpha_i y_i b$ and α_i are constants with respect to θ, and the only θ-dependent piece is $\alpha_i y_i \theta^T x_i$ whose gradient is $\alpha_i y_i x_i$:

$$\nabla_\theta L = \theta - \nabla_\theta \left(\sum_{i=1}^{n} \alpha_i y_i \theta^T x_i + \alpha_i y_i b - \alpha_i\right) = \theta - \sum_{i=1}^{n} \alpha_i y_i x_i = 0$$

Therefore the optimal θ is

$$\boxed{\theta = \sum_{i=1}^{n} \alpha_i y_i x_i}$$

This is a closed form in terms of the data points x_i, their labels y_i (both known) and the multipliers α_i (not yet known). If we knew α, we would know θ.

3.2 Derivative with respect to b

The first term does not contain b, and neither does the last term. Only the middle term does, and its derivative is $\alpha_i y_i$:

$$\frac{\partial L}{\partial b} = 0 - \sum_{i=1}^{n} \alpha_i y_i = 0 \quad \Longrightarrow \quad \boxed{\sum_{i=1}^{n} y_i \alpha_i = 0}$$

3.3 Summary of the KKT conditions

Condition	Meaning
$\theta = \sum_i \alpha_i y_i x_i$	θ is a weighted combination of data points
$\sum_i y_i \alpha_i = 0$	The weighted labels cancel out
$\alpha_i \geq 0$	Dual feasibility

> **Pro-Tip:** Since $\alpha_i = 0$ for all non-support vectors, $\theta = \sum_i \alpha_i y_i x_i$ really only uses the **support vectors**. That is where the name "support vector machine" comes from.

4. From Primal to Dual

We still don't know α. The idea is to **put the two stationarity results back into** L. This removes θ and b completely and leaves a function of α alone. We can then solve for α and recover everything else.

Start from

$$L(\theta, b, \alpha) = \frac{1}{2}\theta^T\theta - \sum_{i=1}^{n} \alpha_i y_i x_i^T \theta - b\sum_{i=1}^{n} \alpha_i y_i + \sum_{i=1}^{n} \alpha_i$$

(We used $\theta^T x_i = x_i^T \theta$ since it is a scalar.)

Step 1: kill the b term. b is a constant with respect to the summation, so it comes outside:

$$b\sum_{i=1}^{n} \alpha_i y_i = b \cdot 0 = 0$$

Step 2: substitute θ. Because i is already used as a running index, we rename the dummy index in θ to j (any letter would do):

$$\theta = \sum_{j=1}^{n} \alpha_j y_j x_j, \qquad \theta^T = \sum_{i=1}^{n} \alpha_i y_i x_i^T$$

The first term becomes (the transpose is taken inside the sum, while α_i and y_i are scalars and stay as they are):

$$\frac{1}{2}\theta^T\theta = \frac{1}{2}\left(\sum_{i=1}^{n} \alpha_i y_i x_i^T\right)\left(\sum_{j=1}^{n} \alpha_j y_j x_j\right) = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j x_i^T x_j$$

The second term becomes

$$\sum_{i=1}^{n} \alpha_i y_i x_i^T \theta = \left(\sum_{i=1}^{n} \alpha_i y_i x_i^T\right)\left(\sum_{j=1}^{n} \alpha_j y_j x_j\right) = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j x_i^T x_j$$

Step 3: combine. Both double sums are the same expression, so one of them is just the other with a factor of $\frac{1}{2}$ less:

$$\begin{aligned} L(\alpha) &= \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j x_i^T x_j - \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^{n} \alpha_i \\ &= -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j x_i^T x_j + \sum_{i=1}^{n} \alpha_i \end{aligned}$$

This is now **purely a function of** α (the y_i and x_i are known data).

The dual problem

$$\max_{\alpha} \quad \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j x_i^T x_j \quad \text{subject to} \quad \alpha_i \geq 0$$

> **Terminology.**
>
> - The original problem in (θ, b) with complicated inequality constraints is the **primal problem**.
> - The new problem in α is the **dual problem**, and α is called the **dual variable**.

> **Note for Review:** The condition $\sum_i y_i\alpha_i = 0$ obtained in §3.2 is still a valid constraint on α. It was used above to eliminate the b term, so a complete statement of the dual problem is $\max_\alpha \ \sum_i \alpha_i - \frac{1}{2}\sum_i\sum_j \alpha_i\alpha_j y_i y_j x_i^T x_j \quad \text{s.t.} \quad \alpha_i \geq 0, \ \sum_i y_i\alpha_i = 0$ In the lecture the box shows only $\alpha_i \geq 0$, but any QP solver for the SVM will be given the equality constraint too.

5. The Dual Problem in Matrix Form

The double summation is quadratic in α, since $\alpha_i \alpha_j$ appears in every term. Define the matrix Q with entries

$$Q_{ij} = -\frac{1}{2} y_i y_j \, x_i^T x_j$$

(this is $-\frac{1}{2}$ times the element-wise product of yy^T with the Gram matrix XX^T). Then the dual becomes

$$\max_{\alpha} \quad \alpha^T Q \, \alpha + \mathbf{1}^T \alpha \quad \text{subject to} \quad I\alpha \geq 0$$

where $\mathbf{1}$ is the vector of all ones and I is the identity matrix (so $I\alpha \geq 0$ just says every $\alpha_i \geq 0$).

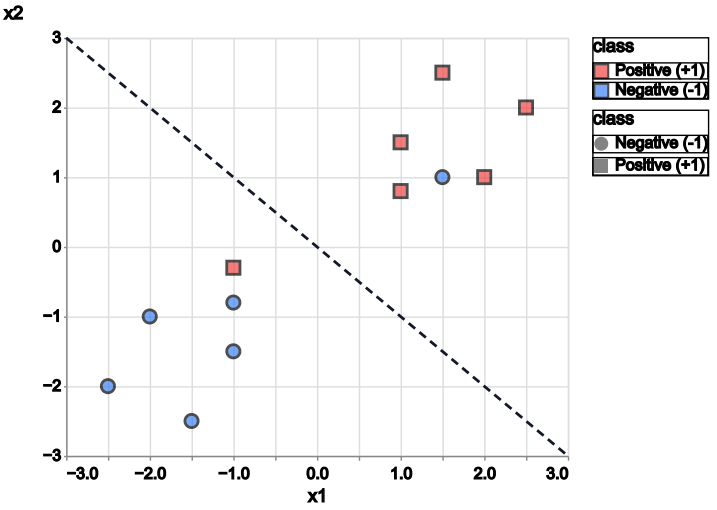
This is a standard **quadratic program**, so any QP library can solve it. Once we have α, the other variables θ and b follow easily.

> **Remark.** Many routes lead to the SVM solution: the Lagrange multiplier route above, or writing the primal directly as a quadratic program (seen earlier). Both end in a QP. In a live session, this entire SVM will be coded and demonstrated.

6. Non-Separable Classes with Slack

When classes are not perfectly separable (e.g. a few outliers cross the boundary) we introduce **slack variables** \eta_i, and allow only some points to violate the margin.

*The dashed line is the linear boundary $x_1 + x_2 = 0$. One positive point and one negative point lie on the wrong side. These are the outliers, and they are exactly the points with $\eta_i > 0$.



6.1 Primal problem with slack

$$\min_{\theta, b, \eta} \quad \frac{1}{2}\theta^T \theta + C \sum_{i=1}^{n} \eta_i$$

$$\text{subject to} \quad y_i(\theta^T x_i + b) \geq 1 - \eta_i, \quad i = 1, \ldots, n$$

$$\eta_i \geq 0, \quad i = 1, \ldots, n$$

- η_i is the **margin by which a point is allowed to go beyond the support vector line**. It is a positive quantity.
- If only a few points cause the non-separability, η_i is non-zero for just those few points and $\eta_i = 0$ for most of the others.
- C is a constant that controls the penalty. We minimize $\sum \eta_i$ so that slack is not handed out freely.
- The η_i are also unknown, so they are now an extra set of optimization variables.

6.2 Lagrangian

We now have **two sets of inequality constraints**, so we need two sets of multipliers: α_i for the margin constraints and β_i for the positivity of η_i:

$$L(\theta, b, \alpha, \beta, \eta) = \frac{1}{2}\theta^T\theta + C\sum_{i=1}^{n}\eta_i - \sum_{i=1}^{n}\alpha_i(y_i(\theta^T x_i + b) - 1 + \eta_i) - \sum_{i=1}^{n}\beta_i\eta_i$$

with KKT feasibility conditions

$$\alpha_i \geq 0, \qquad \beta_i \geq 0, \qquad i = 1, \ldots, n$$

6.3 Continuing the derivation (same recipe)

Setting the gradients to zero gives the same two conditions as before, plus one for η_i:

$$\theta = \sum_{i=1}^{n}\alpha_i y_i x_i, \qquad \sum_{i=1}^{n}\alpha_i y_i = 0, \qquad \frac{\partial L}{\partial \eta_i} = C - \alpha_i - \beta_i = 0$$

When we substitute back, the η terms collect as

$$C\sum_i \eta_i - \sum_i \alpha_i\eta_i - \sum_i \beta_i\eta_i = \sum_i (C - \alpha_i - \beta_i)\,\eta_i = 0$$

so they disappear, and we get **exactly the same dual objective** as the separable case. The only change is the constraint: since $\beta_i \geq 0$ and $\beta_i = C - \alpha_i$, we need

$$0 \leq \alpha_i \leq C$$

> **Pro-Tip:** Compare the two duals. Separable: $\alpha_i \geq 0$. Non-separable: $0 \leq \alpha_i \leq C$. The slack formulation simply puts an **upper cap** on how much any single point can pull the boundary. A frequent exam question is "what happens as $C \to \infty$?" (the cap vanishes and we get back the hard-margin SVM).

---

7. Recovering Parameters and Predicting

Once a QP solver returns α:

1. **Weights:** $\theta = \sum_{i=1}^{n}\alpha_i y_i x_i$ (only support vectors contribute).
2. **Bias:** pick any support vector x_s with $\alpha_s > 0$. It lies exactly on the margin, so $y_s(\theta^T x_s + b) = 1$. Since $y_s = \pm 1$, this gives

$$b = y_s - \theta^T x_s$$

3. **Prediction** for a new point x:

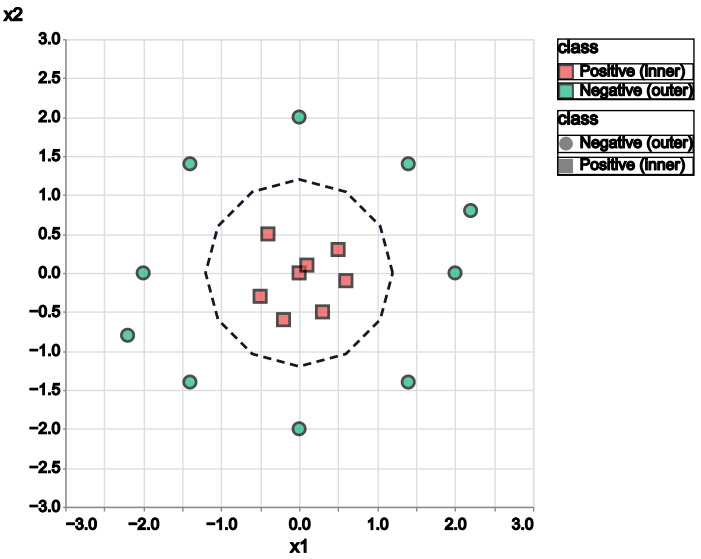
$$h(x) = \operatorname{sgn}\left(\sum_{i=1}^{n}\alpha_i y_i\, x_i^T x + b\right)$$

Notice that, in both training (the dual objective) and prediction, the data only ever appear through the inner products $x_i^T x_j$ and $x_i^T x$. This single observation is the doorway to the kernel trick.

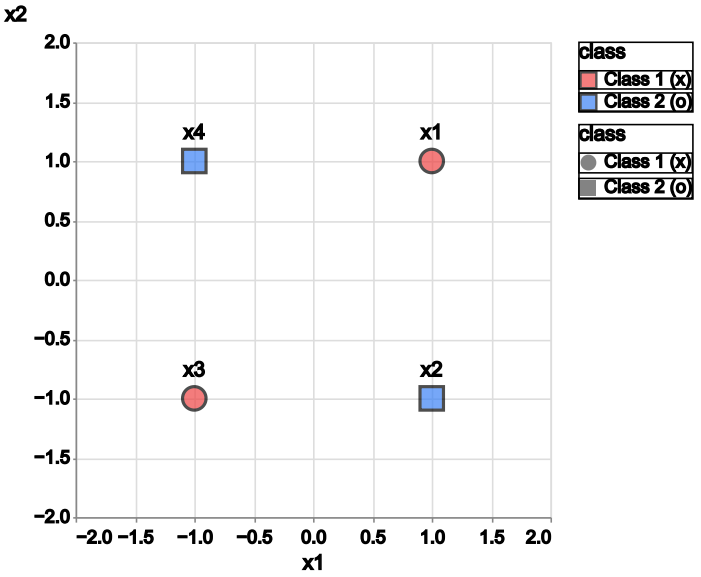
8. When a Line Is Not Enough

So far the two classes could be separated by a linear boundary (with outliers handled by slack). Now consider data where the **true boundary is non-linear**.

Example A: one class enclosed by the other. Positive points (crosses) sit in the middle, and negative points (circles) surround them. The decision boundary is a circle, and no straight line can do the job.



Example B: the XOR pattern. The simplest non-separable data set has only four points. Two from class 1 sit at opposite corners, and two from class 2 sit at the other two corners. Any boundary that separates them has to bend (it looks like a cross or a pair of curves).



Point	Coordinates	Class
x_1	$(1, 1)$	×
x_2	$(1, -1)$	○
x_3	$(-1, -1)$	×
x_4	$(-1, 1)$	○

Linear classifiers (logistic regression, SVM) cannot solve these problems directly. Later in the course, **neural networks with non-linear activation functions** will learn such boundaries naturally. Before neural networks became popular, a clever trick was used instead.

9. Lifting Data to Higher Dimensions

Idea: map every data point to a **higher-dimensional space**, where the classes become linearly separable, and then run the ordinary linear SVM there.

Define a mapping ϕ that consumes x_i and produces $\bar{x}_i$:

$$\phi(x_i) = \bar{x}_i, \qquad x_i \in \mathrm{R}^2, \quad \bar{x}_i \in \mathrm{R}^d, \quad d > 2$$

A simple lifting example

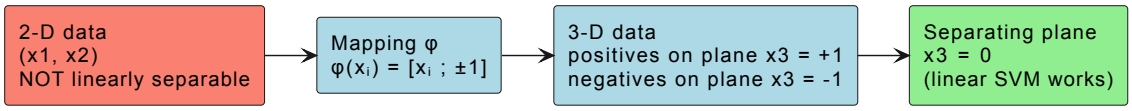
For the ring data of Example A, append an extra coordinate that depends on the label:

$$\phi(x_i) = \bar{x}_i = \begin{cases} \begin{bmatrix} x_i \\ 1 \end{bmatrix} & \text{for all } y_i = 1 \\ \begin{bmatrix} x_i \\ -1 \end{bmatrix} & \text{for all } y_i = -1 \end{cases}$$

Now the points live in 3-D with axes x_1, x_2, x_3:

- All **positive** points have $x_3 = +1$, so they lie on the plane $x_3 = 1$.
- All **negative** points have $x_3 = -1$, so they lie on the plane $x_3 = -1$.
- x_1, x_2 are unchanged.

The two classes sit on two different parallel planes, so a plane (for example $x_3 = 0$) separates them. A linear SVM in 3-D solves the problem.



> **Note for Review:** This particular ϕ uses the label y_i, so it can't be applied to a new, unlabelled test point. It is only an illustration of *why* lifting helps. Genuine feature maps depend on x alone, as in §11. Finding a good ϕ in general is a separate problem that comes later in the course.

10. Kernel Functions

Let $\mathrm{X} = \{x_1, \ldots, x_n\}$ be the set of data points (only the points, not the labels). A **kernel** is a function that takes two data points and produces a scalar:

$$K : \mathrm{X} \times \mathrm{X} \to \mathrm{R}$$

Why do we care?

Look back at the dual problem. To find α we do **not** need the individual data points; we only need the **pairwise inner products** $x_i^T x_j$. The inner product is a measure of **similarity**:

- it is large and positive when two vectors point the same way (for unit vectors, equal to 1 if they are identical),
- it is 0 if they are perpendicular,
- it is negative (for unit vectors, -1) if they point in opposite directions.

The inner product is itself a function with two arguments that returns a scalar:

$$\langle \cdot, \cdot \rangle : \mathrm{X} \times \mathrm{X} \to \mathrm{R}, \qquad \langle x_i, x_j \rangle = x_i^T x_j$$

So the ordinary inner product **is already a (very simple) kernel**:

$$K(x_i, x_j) = \langle x_i, x_j \rangle$$

The generalized definition of a kernel is the same idea: any function that takes two data points and returns a similarity score. In the dual we can therefore replace $x_i^T x_j$ by $K(x_i, x_j)$.

11. The Kernel Trick

11.1 The set-up

Suppose x_i and x_j are points in the original space, and ϕ maps them to a higher-dimensional space as $\phi(x_i)$ and $\phi(x_j)$. After lifting, the similarity the SVM needs is the one **in the high-dimensional space**:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

The key realization: **we never actually need ϕ itself**. We only need a way to compute the similarity score in the higher-dimensional space. If we can write down that similarity directly as a formula in x and z, we skip the mapping.

(To avoid subscript clutter, use x and z for two data points.)

Take, as a particular choice, the similarity

$$K(x, z) = (x^T z + 1)^2$$

Question: is this really an inner product $\langle \phi(x), \phi(z) \rangle$ in some higher-dimensional space? We prove that it is, and in doing so find the ϕ.

11.2 Expanding the polynomial kernel

Let the data live in R^2: $x = (x_1, x_2)$ and $z = (z_1, z_2)$. Then

$$(x^T z + 1)^2 = (x_1 z_1 + x_2 z_2 + 1)^2$$

Squaring a sum of three terms gives the three squares plus twice every cross product:

$$= x_1^2 z_1^2 + x_2^2 z_2^2 + 1 + 2x_1 z_1 x_2 z_2 + 2x_1 z_1 + 2x_2 z_2$$

This is a scalar, and it is non-negative (it is a square), as a similarity should be. Now split each term into an "x-part" times a "z-part", inserting $\sqrt{2}$ so that $\sqrt{2} \cdot \sqrt{2} = 2$:

$$= x_1^2 \cdot z_1^2 \;+\; x_2^2 \cdot z_2^2 \;+\; \sqrt{2}x_1 x_2 \cdot \sqrt{2} z_1 z_2 \;+\; \sqrt{2}x_1 \cdot \sqrt{2} z_1 \;+\; \sqrt{2}x_2 \cdot \sqrt{2} z_2 \;+\; 1 \cdot 1$$

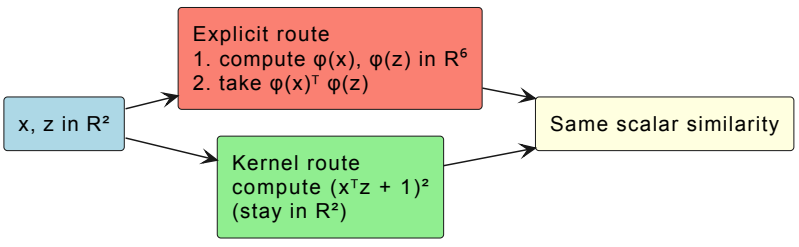
This is an inner product of two 6-dimensional vectors:

$$(x^T z + 1)^2 = \begin{bmatrix} x_1^2 & x_2^2 & \sqrt{2}x_1 x_2 & \sqrt{2}x_1 & \sqrt{2}x_2 & 1 \end{bmatrix} \begin{bmatrix} z_1^2 \\ z_2^2 \\ \sqrt{2}z_1 z_2 \\ \sqrt{2}z_1 \\ \sqrt{2}z_2 \\ 1 \end{bmatrix} = \phi(x)^T \phi(z)$$

So the feature map is

$$\phi : x \mapsto \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1 x_2 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ 1 \end{bmatrix}, \qquad \phi : \mathrm{R}^2 \to \mathrm{R}^6$$

We have verified two things: the kernel really is a similarity in a higher-dimensional space, and that space is R^6.



11.3 Verifying on the XOR data

Using $\phi(x) = (x_1^2,\ x_2^2,\ \sqrt{2}x_1x_2,\ \sqrt{2}x_1,\ \sqrt{2}x_2,\ 1)$, compute the image of each of the four points, one coordinate at a time.

$x_1 = (1, 1)$:

$$\phi(x_1) = (1^2,\ 1^2,\ \sqrt{2}\cdot 1\cdot 1,\ \sqrt{2}\cdot 1,\ \sqrt{2}\cdot 1,\ 1) = (1, 1, \sqrt{2}, \sqrt{2}, \sqrt{2}, 1)$$

$x_2 = (1, -1)$:

$$\phi(x_2) = (1,\ 1,\ \sqrt{2}\cdot (1)(-1),\ \sqrt{2}\cdot 1,\ \sqrt{2}\cdot (-1),\ 1) = (1, 1, -\sqrt{2}, \sqrt{2}, -\sqrt{2}, 1)$$

$x_3 = (-1, -1)$:

$$\phi(x_3) = (1,\ 1,\ \sqrt{2}\cdot (-1)(-1),\ \sqrt{2}\cdot (-1),\ \sqrt{2}\cdot (-1),\ 1) = (1, 1, \sqrt{2}, -\sqrt{2}, -\sqrt{2}, 1)$$

$x_4 = (-1, 1)$:

$$\phi(x_4) = (1,\ 1,\ \sqrt{2}\cdot (-1)(1),\ \sqrt{2}\cdot (-1),\ \sqrt{2}\cdot 1,\ 1) = (1, 1, -\sqrt{2}, -\sqrt{2}, \sqrt{2}, 1)$$

Point	Class	$\phi(x)$	3rd coordinate $\sqrt{2}x_1x_2$
x_1	×	$(1, 1, \sqrt{2}, \sqrt{2}, \sqrt{2}, 1)$	$+\sqrt{2}$
x_3	×	$(1, 1, \sqrt{2}, -\sqrt{2}, -\sqrt{2}, 1)$	$+\sqrt{2}$
x_2	○	$(1, 1, -\sqrt{2}, \sqrt{2}, -\sqrt{2}, 1)$	$-\sqrt{2}$
x_4	○	$(1, 1, -\sqrt{2}, -\sqrt{2}, \sqrt{2}, 1)$	$-\sqrt{2}$

The classes are now linearly separable. Both × points have third coordinate $+\sqrt{2}$ and both ○ points have $-\sqrt{2}$, so the hyperplane "third coordinate $= 0$" (for instance $\theta = (0, 0, \frac{1}{\sqrt{2}}, 0, 0, 0)$, $b = 0$, which gives $y_i(\theta^T \phi(x_i)) = 1$ exactly) separates them. The XOR problem, impossible in R^2, is trivial in R^6.

The lifted coordinate x_1x_2 is precisely the product that XOR depends on: it is positive when the signs agree and negative when they differ.

Checking that the kernel agrees with the explicit inner product.

Using the kernel directly, $K(x_1, x_2) = (x_1^T x_2 + 1)^2$. Here $x_1^T x_2 = (1)(1) + (1)(-1) = 0$, so

$$K(x_1, x_2) = (0 + 1)^2 = 1$$

Using the explicit map:

$$\phi(x_1)^T \phi(x_2) = 1\cdot 1 + 1\cdot 1 + \sqrt{2}\cdot(-\sqrt{2}) + \sqrt{2}\cdot\sqrt{2} + \sqrt{2}\cdot(-\sqrt{2}) + 1\cdot 1 = 1 + 1 - 2 + 2 - 2 + 1 = 1 \checkmark$$

And the diagonal case: $x_1^T x_1 = 2$, so $K(x_1, x_1) = (2 + 1)^2 = 9$, while

$$\phi(x_1)^T \phi(x_1) = 1 + 1 + 2 + 2 + 2 + 1 = 9 \checkmark$$

Doing this for every pair gives the full kernel (Gram) matrix, with rows and columns ordered x_1, x_2, x_3, x_4:

$$K = \begin{bmatrix} 9 & 1 & 1 & 1 \\ 1 & 9 & 1 & 1 \\ 1 & 1 & 9 & 1 \\ 1 & 1 & 1 & 9 \end{bmatrix}$$

(For example $x_1^T x_3 = -2$, so $K(x_1, x_3) = (-2 + 1)^2 = 1$, and $x_2^T x_4 = -2$, so $K(x_2, x_4) = 1$ as well.)

11.4 Why this is a "trick"

Without ever writing down ϕ, we can replace $x_i^T x_j$ in the SVM by $(x_i^T x_j + 1)^2$ and the algorithm automatically finds the **linear boundary in the higher-dimensional space**. That corresponds to a **non-linear boundary in the original space**. All the work needs only the kernel formula, not the mapping ϕ itself.

> **Pro-Tip:** Computing $(x^T z + 1)^2$ costs about d multiplications in R^d, while building the explicit degree-2 feature vector costs roughly $d^2/2$ coordinates. For high-degree polynomials the gap becomes enormous. This computational saving is the real reason the trick is useful.

12. Kernels Inside the SVM Dual

Go back to the dual problem. Wherever $x_i^T x_j = \langle x_i, x_j \rangle$ appears, replace it by the kernel. With the polynomial kernel of §11 the replacement chain is

$$x_i^T x_j \;\longrightarrow\; \langle x_i, x_j \rangle \;\longrightarrow\; K(x_i, x_j) \;\longrightarrow\; (\langle x_i, x_j \rangle + 1)^2$$

so the dual problem becomes

$$\max_{\alpha} \; \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, K(x_i, x_j) \quad \text{subject to} \quad \alpha_i \geq 0, \;\; \sum_i y_i \alpha_i = 0$$

and, using the same replacement in §7, the prediction rule is

$$h(x) = \operatorname{sgn}\left(\sum_{i=1}^{n} \alpha_i y_i \, K(x_i, x) + b \right)$$

Nothing else in the algorithm changes. The solution corresponds to a linear separator in the higher-dimensional space, even though that space never appears explicitly.

> **Note on notation:** In the lecture the kernel is written as $K(\phi(x_i), \phi(x_j))$ to emphasize "similarity after mapping". In practice (and in the formulas above) one writes $K(x_i, x_j)$, which is *defined* to equal $\phi(x_i)^T \phi(x_j)$.

13. The Gaussian Kernel

We can define a kernel without knowing (or even having a finite) ϕ at all. The most famous example is the **Gaussian kernel**:

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2} \right)$$

How it behaves:

Situation	$\|x_i - x_j\|$	$K(x_i, x_j)$
Points are identical	0	1 (maximum similarity)
Points are close	small	close to 1
Points are far apart	large	close to 0

So it is a proper similarity measure: **high for nearby points, low for distant ones**. We can plug it into the SVM dual of §12 and solve the non-linearly separable problem **without ever knowing the mapping ϕ**.

This is why it is called the **kernel trick**: we solve the non-linear problem by working implicitly in a higher-dimensional space, without ever computing the map to that space.

> **Pro-Tip:** The feature space of the Gaussian kernel is actually **infinite-dimensional**, so an explicit ϕ could never be written down. This is the strongest argument for the trick: the kernel is cheap to compute even when ϕ is impossible to store. The width σ controls how quickly similarity fades with distance.

Coming next: more examples of kernels, and the precise conditions a function must satisfy to be a valid kernel.

14. Practice Questions

1. **Support vectors and the dual.** In the dual solution, $\theta = \sum_i \alpha_i y_i x_i$. Suppose you delete a training point that has $\alpha_i = 0$ and re-train. What happens to θ and b, and why? Now delete a point with $\alpha_i > 0$. Construct a small 2-D example where this visibly moves the boundary.

2. **Reading the constraint $\sum_i y_i \alpha_i = 0$.** Imagine a data set with 10 positive points and 1 negative point. Using only this constraint and $\alpha_i \geq 0$, argue what the multiplier of the lone negative point must look like compared with the multipliers of the positives. Which class "pays" for the imbalance?

3. **Dual with slack.** In the soft-margin dual the multipliers satisfy $0 \leq \alpha_i \leq C$. Consider three points: one far inside the correct side, one exactly on the margin, and one that is misclassified. Which of the three would you expect to have $\alpha_i = 0$, $0 < \alpha_i < C$, and $\alpha_i = C$? What does η_i look like for each?

4. **A different kernel by hand.** Take $x, z \in \mathrm{R}^2$ and the kernel $K(x, z) = (x^T z)^2$ (no "+1"). Expand it as you did for $(x^T z + 1)^2$ and find an explicit ϕ and the dimension of its target space. Then explain which of the six coordinates of the earlier map you have lost, and what kind of decision boundaries you can no longer represent (hint: think about whether a boundary through the origin versus off the origin is possible).

5. **Does the XOR trick need all six coordinates?** In §11.3 the separating hyperplane only used the third coordinate $\sqrt{2}x_1 x_2$. Design a *different* feature map $\psi : \mathrm{R}^2 \to \mathrm{R}^3$ that also makes XOR linearly separable, and write the kernel $K(x, z) = \psi(x)^T \psi(z)$ it induces. Then compare the cost of computing this kernel with the cost of $(x^T z + 1)^2$, and discuss why, for realistic data that is not as symmetric as XOR, the richer six-dimensional map might still be preferable.